

От Legacy к V6: Архитектурное проектирование LocalPDF как системы на основе реестра, воркеров и декларативных ограничений

Архитектура ядра: Глобальный Реестр и Плагинная Модель

Данная глава посвящена детальному анализу и проектированию двух фундаментальных компонентов архитектуры следующего поколения LocalPDF (V6): глобального реестра и гибридной плагинной архитектуры. Эти элементы формируют основу для создания масштабируемого, расширяемого и легко поддерживаемого приложения, решая ключевые проблемы устаревшей Legacy-системы и экспериментальной V5-версии. Legacy-архитектура характеризовалась жёсткой привязкой маршрутизации к конкретным инструментам через конструкцию `switch` в файле `App.tsx` и использованием карты `TOOL_HASH_MAP` [67](#). Такой подход делал систему хрупкой и трудно расширяемой, поскольку добавление нового инструмента требовало внесения изменений в код маршрутизации и меню навигации. Экспериментальная V5-версия заложила основу для решения этих проблем, внедрив понятия `toolRegistry` и `V5UnifiedToolRunner` [1](#), однако не решила всех фундаментальных вопросов модульности и изоляции. Новая архитектура V6 направлена на кардинальное устранение этих недостатков путём создания единого источника истины и разделения ответственности между пользовательским интерфейсом и бизнес-логикой.

Центральным элементом новой архитектуры является Глобальный Реестр, который должен стать единственным источником истины для роутинга, построения пользовательского интерфейса (в частности, боковой панели навигации) и управления правами доступа. Это напрямую противодействует хаосу, вызванному жёстким роутингом в Legacy-версии. Реестр должен представлять собой централизованное хранилище метаданных, которое декларативно описывает каждый инструмент, его возможности и требования к

доступу. На практике это может быть реализовано как набор TypeScript-интерфейсов или JSON-файлов, которые собираются воедино в процессе сборки приложения. Такой подход позволяет полностью автоматизировать процессы, ранее требовавшие ручного кодирования. Например, вместо статических путей в React Router, система сможет динамически генерировать `<Route>` элементы на основе данных, полученных из реестра. Это достигается с помощью высшего порядка компонента (НОС) или кастомного хука, который запрашивает список инструментов из реестра и рендерит соответствующие маршруты. Аналогично, компоненты, отвечающие за навигацию, становятся чистыми представлениями, которые принимают массив инструментов из реестра и отрисовывают их в виде списка или иконок. Это гарантирует, что любое добавление или удаление инструмента автоматически отражается в интерфейсе без необходимости вносить изменения в код самих компонентов навигации.

Ключевой особенностью Global Registry является его роль в управлении правами доступа. Вместо того чтобы размазывать проверки прав (`if (isFree) { ... }`) по всему коду сервисов, как это было замечено в текущей реализации, права доступа должны быть декларативно заданы непосредственно в определении инструмента. Реестр должен содержать информацию о правах доступа (RBAC), например, в виде массива строк, таких как `entitlements: ["pdf.merge", "pdf.split"]`. Это позволяет централизованно управлять доступом через механизм `EntitlementGate`, как это было начато в V5, но без необходимости размазывать проверки по всему коду [61](#) [63](#). Когда пользователь пытается получить доступ к инструменту, система обращается к реестру, чтобы узнать необходимые права, и затем использует сервис прав доступа для проверки наличия этих прав у текущего пользователя. Этот подход значительно повышает читаемость и поддерживаемость кода, так как логика доступа отделена от бизнес-логики самого инструмента. Для реализации этого можно использовать стандартные модели авторизации, такие как ACL (управление списками контроля доступа), RBAC (основанное на роли контроль доступа) или ABAC (управление доступом на основе атрибутов) [66](#).

Связанным с этим является плагиновая архитектура, которая предполагает гибридный подход к модульности. Пользовательский интерфейс каждого инструмента должен быть реализован как тонкий ES-модуль, который загружается лениво с помощью механизма `React.lazy` [22](#) [80](#). Это стандартная практика для оптимизации производительности в современных веб-

приложениях, особенно когда количество компонентов велико, как в случае с каталогом из 21 инструмента [67](#). Ленивая загрузка позволяет приложению не загружать код всех инструментов сразу, что могло бы привести к увеличению размера основного JavaScript-пакета и замедлению первоначальной загрузки страницы. При переходе пользователя на инструмент «Объединить PDF», система динамически импортирует только компонент **MergeToolUI**. После того как этот компонент был отмонтирован со страницы, он становится кандидатом для сборки мусора, освобождая ресурсы. Это достигается за счет использования динамических импортов (`dynamic import()`) в сочетании с возможностями дерева шейки [34](#).

Однако, согласно запросу, этого недостаточно. Бизнес-логика, связанная с обработкой файлов (например, слияние страниц с помощью `pdf-lib`, сжатие, OCR), должна выполняться в полной изоляции от основного потока. Поэтому вторая часть плагиновой архитектуры — это строгая изоляция бизнес-логики в **Web Workers** [100](#). Это решение критически важно для предотвращения блокировки пользовательского интерфейса при выполнении ресурсоёмких операций, что является одной из главных проблем Legacy-версии. Операции, такие как копирование страниц или анализ и пересборка PDF-документов, сейчас выполняются в основном потоке или частично через промисы, что приводит к зависанию интерфейса при работе с большими файлами [59](#) [107](#). Хотя в V5 уже используется `ocrWorkerManager` для задач OCR, это скорее исключение, чем правило. В рамках V6 любой процесс обработки, даже простой **MergeService**, должен быть запущен в отдельном фоновом потоке. Это гарантирует, что пользовательский интерфейс всегда остается отзывчивым, позволяя ему выполнять другие действия или даже переключаться между вкладками браузера, пока идёт длительная обработка.

Таким образом, плагиновая архитектура V6 представляет собой связку из двух взаимодополняющих моделей: 1. **UI-плагины**: Лениво загружаемые ES-модули, содержащие React-компоненты. Они отвечают только за отображение интерфейса настроек и передачу данных. 2. **Логические плагины**: Код, предназначенный для выполнения в **Web Worker**. Он содержит основную бизнес-логику инструмента и экспортирует одну или несколько функций, принимающих входные данные (например, массив **Blob**) и возвращающих результат (также **Blob**). Сам инструмент («OCR», «Split») в этой модели — это не просто один файл, а комплексное решение, состоящее из UI-компонента и логического плагина.

Эта архитектура обеспечивает чёткое разделение ответственности: пользовательский интерфейс отвечает за взаимодействие с пользователем, а логические плагины — за тяжёлые вычисления. Это не только улучшает производительность, но и повышает безопасность и предсказуемость системы, так как изоляция в воркерах предотвращает побочные эффекты между разными инструментами (например, случайное изменение глобального состояния одним инструментом не должно влиять на работу другого).

Для организации такого модульного проекта предлагается следующая структура каталогов. Корневой каталог `src/plugins` будет содержать папки для каждого инструмента. Каждая папка инструмента, например `src/plugins/merge-pdf`, будет иметь следующую внутреннюю структуру:

```
src/
├─ plugins/
│   └─ [tool-name]/
│       ├─ ui/
│       │   └─ index.tsx           # Экспорт React-компонента UI
│       ├─ logic/
│       │   └─ index.ts           # Экспорт функции для Web Worker
│       └─ definition.ts          # Определение инструмента для GlobalRegistry
└─ ...
```

Система плагинов будет сканировать каталог `src/plugins`, находить папки инструментов, импортировать их `definition.ts` файлы и регистрировать все инструменты в центральном `GlobalRegistry`. Это позволит системе автоматически собирать список всех доступных инструментов и их метаданных без необходимости ручного добавления каждого нового инструмента в какой-либо центральный файл регистрации. Такой подход соответствует принципам Clean Architecture, где доменная логика (в данном случае, бизнес-логика инструментов) отделена от инфраструктурных деталей (загрузка плагинов, маршрутизация) [16](#). Это создаёт прочную основу для будущего развития приложения, позволяя легко добавлять новые функции и инструменты, не затрагивая существующий код маршрутизации и навигации.

Компонент	Legacy / V5	V6 Архитектура
Роутинг	Жёсткий роутинг (switch в App.tsx) и карта TOOL_HASH_MAP.	Декларативный роутинг на основе GlobalRegistry.
Меню/Навигация	Ручное управление состоянием, зависимость от currentTool.	Чистый компонент, отрисовывающийся на основе данных из GlobalRegistry.
Загрузка UI	Все компоненты инструментов загружаются вместе с основным пакетом.	Ленивая загрузка UI-компонентов через ES-модули (React.lazy).
Выполнение логики	Прямые вызовы тяжёлых библиотек (pdf-lib, tesseract.js) в основном потоке.	Изоляция бизнес-логики в Web Workers для предотвращения блокировки UI.
Добавление инструмента	Требует изменений в нескольких местах (роутинг, меню, состояние).	Требует создания новой плагин-структуры; система сама регистрирует инструмент.

Эта таблица наглядно демонстрирует прогрессию и преимущества предлагаемой архитектуры V6. Она не просто улучшает старые подходы, а предлагает фундаментально новый способ построения приложения, основанный на принципах декларативности, модульности и изоляции.

Изоляция Вычислений и Управление Потоками

Критическим аспектом производительности любого приложения, работающего с ресурсоёмкими данными, таких как PDF-документы и изображения, является избегание блокировки основного потока браузера. В Legacy-версии LocalPDF операции, такие как слияние страниц с помощью библиотеки pdf-lib или применение OCR через tesseract.js, выполнялись либо напрямую в основном потоке, либо через простые промисы, что часто приводило к зависанию интерфейса при обработке больших файлов [102](#). Хотя Tauri-версия может предлагать преимущество за счёт более быстрой работы нативных библиотек, эта фундаментальная проблема сохраняется, если логика выполняется в том же потоке, что и пользовательский интерфейс. Запрос пользователя на обязательную изоляцию бизнес-логики в Web Workers является ключевым решением этой проблемы. Однако простого запуска кода в воркере недостаточно; необходимо создать надлежащий уровень абстракции и стандартизировать протокол взаимодействия между основным потоком (пользовательским интерфейсом) и рабочими процессами. Именно для этого предназначен Оркестратор воркеров.

Worker Orchestrator — это слой абстракции, который берёт на себя всю сложность управления жизненным циклом Web Workers. Его основные обязанности включают: 1. Запуск и остановка воркеров: Оркестратор управляет пулом воркеров или создает их по мере необходимости для выполнения задач. Это позволяет эффективно использовать ресурсы, не создавая новых воркеров для каждой мелкой операции. 2. Инкапсуляция взаимодействия: Он скрывает от остальной части приложения детали использования **Worker**, **postMessage** и **onmessage**. UI-компоненты и Runner'ы взаимодействуют с оркестратором через простой и понятный API, а не напрямую с объектами DOM. 3. Обработка сообщений: Оркестратор парсит входящие сообщения, определяет тип команды и передаёт её соответствующему обработчику (обычно это **UnifiedToolRunner**). 4. Передача данных: Он отвечает за эффективную передачу данных, в частности больших Blob-объектов, между основным потоком и воркером.

Для обеспечения надёжной и предсказуемой коммуникации между UI и воркерами необходимо внедрить паттерн **Command/Event**. Этот паттерн декларирует строгий формат обмена сообщениями, который разделяет команды (инструкции для исполнителя) от событий (уведомления об изменениях состояния).

Протокол обмена сообщениями:

- От UI к Воркеру (**Command Pattern**): Когда пользователь инициирует действие (например, нажимает кнопку «Обработать»), UI отправляет в оркестратор команду. Команда представляет собой JSON-объект с чётко определённой структурой.

```
````typescript
interface ICommand {
 id: string; // Уникальный идентификатор команды для отслеживания
 type: 'PROCESS_TOOL' | 'CREATE_TEMP_FILE' | 'READ_FILE';
 payload: any; // Данные, специфичные для команды (ID файлов, параметры и т.д.)
}
````
```

Пример команды для запуска инструмента: `{ id: 'cmd-123', type: 'PROCESS_TOOL', payload: { ... } }`

- От Воркера к UI (**Event Pattern**): Воркер, выполнив задачу или пройдя определённый этап, отправляет обратно событие. Это может быть событие о прогрессе, результате или ошибке.

```

```typescript
interface IEvent {
 id: string; // ID команды, которую эта ошибка или событие завершило
 type: 'TOOL_PROGRESS' | 'TOOL_RESULT' | 'TOOL_ERROR';
 payload: {
 progress?: number; // От 0 до 100
 outputFiles?: string[]; // UUID новых файлов
 error?: string;
 };
}
```

```

Пример события успешного завершения: `{ id: 'cmd-123', type: 'TOOL_RESULT' }

Этот протокол гарантирует, что основной поток никогда не будет заблокирован операциями, которые могут занять значительное время, такими как обработка больших файлов. Весь тяжёлый труд выполняется в фоновом воркере, а основной поток остаётся свободным для обновления пользовательского интерфейса и обработки пользовательского ввода [28 100](#).

Особое внимание следует уделить эффективной передаче больших данных, таких как Blob-объекты файлов, между основным потоком и воркером. По умолчанию метод `postMessage` использует алгоритм структурированного клонирования, который копирует данные, что может быть очень медленным для больших объектов [103](#). Для оптимизации этого процесса следует использовать передаваемые объекты. Если воркер не нуждается в оригинальном объекте после его передачи, владение буфером данных можно переместить, а не скопировать. Это позволяет передавать большие массивы байтов, такие как Blobs или ArrayBuffer, с минимальными накладными расходами [90](#). Для этого в `postMessage` можно передать второй аргумент, указывающий на массив передаваемых объектов.

```

// Пример передачи Blob без копирования
const blob = /<em> ... </em>/;
worker.postMessage({ type: 'PROCESS_FILE', fileData: blob }, [blob]);

```

В этом случае браузер перемещает владение буфером данных в воркер. После такой передачи попытка использовать тот же `blob` в основном потоке приведёт к ошибке. Этот механизм является ключевым для достижения высокой производительности при обработке больших файлов. Однако, если Blob требуется нескольким частям приложения (например, для предварительного

просмотра и последующей обработки), его нужно передавать обычным путём, но при этом необходимо тщательно управлять его жизненным циклом, чтобы избежать утечек памяти. Здесь на помощь приходит Virtual File System (VFS), которая будет рассмотрена в следующей главе.

В контексте Node.js, аналогом Web Workers являются **Worker Threads**. Они также предназначены для выполнения CPU-ёмких задач в фоновых потоках, не блокируя основной цикл событий [59](#) [91](#). Хотя основной фокус V6 — это веб-версия, использование Worker Threads в Tauri-приложении для тех же целей (обработка больших файлов) будет следовать тем же принципам и паттернам.

Для отладки и тестирования логики, выполняемой в воркерах, могут использоваться различные инструменты. Например, `worker-loader` для `webpack` позволяет преобразовывать код воркера в Blob URL, что упрощает его запуск [40](#). Для локального разработки и тестирования кода Cloudflare Workers можно использовать Miniflare [60](#). Хотя это относится к другому контексту, идея локального окружения для серверных функций применима и здесь. Можно создать среду, имитирующую окружение воркера, для изоляции и проверки бизнес-логики.

Таким образом, создание слоя оркестратора воркеров с чётким протоколом обмена сообщениями и использованием передаваемых объектов для передачи больших данных является решающим шагом для достижения высокой производительности и отзывчивости интерфейса в V6. Это позволяет изолировать ресурсоёмкие вычисления, не усложняя при этом взаимодействие между компонентами приложения.

Виртуальная Файловая Система (VFS) и Абстракция Ввода-Вывода

Фундаментальная проблема, с которой сталкивается приложение LocalPDF, заключается в ручном управлении жизненным циклом файлов и ссылками на них, что приводит к серьёзным утечкам памяти. В коде были обнаружены случаи использования `URL.createObjectURL` без последующего вызова `URL.revokeObjectURL`, что является распространённой причиной утечек памяти в веб-приложениях [12](#) [38](#). Когда `createObjectURL` вызывается для Blob

или `File`, браузер создаёт временную URL-адрес, которая ссылается на данные объекта. Пока хотя бы одна такая ссылка существует, браузер не может собрать мусор для оригинального объекта `Blob`, что может привести к значительному потреблению памяти, особенно при активном использовании больших файлов [39](#). Текущая реализация, где файл хранится в одном месте (`useSharedFile`), делает невозможными сложные сценарии обработки, требующие временных промежуточных результатов без повторной загрузки и сохранения [13](#). Чтобы решить эту проблему коренным образом, предлагается внедрить виртуальную файловую систему (**VFS**).

Виртуальная файловая система (**VFS**) — это слой абстракции, который предоставляет единый, унифицированный интерфейс для работы с файлами, независимо от того, где они физически хранятся. Ключевым элементом VFS является интерфейс `IFileEntry`, который инкапсулирует информацию о файле и предоставляет методы для взаимодействия с ним, например, `.getBlob()`, `.getText()` или `.getSize()`. Инструменты приложения работают с объектами `IFileEntry`, а не напрямую с `Blob` или путями к файлам, что позволяет им быть полностью агностичными к механизму хранения [20](#). Это решает проблему, когда инструмент должен одинаковым образом обрабатывать как файл, загруженный пользователем, так и результат работы предыдущего инструмента в цепочке обработки.

Центральным принципом VFS является разделение стратегий хранения на основе типа файла. Это позволяет применять оптимальные подходы к управлению каждым типом данных:

1. Стратегия для пользовательских загрузок (**UserUploadStrategy**): Файлы, которые пользователь выбирает вручную, должны храниться надёжно, чтобы пережить перезагрузку страницы или закрытие браузера. Для этого рекомендуется использовать долговременное хранилище, такое как **IndexedDB** в веб-версии или прямой доступ к нативной файловой системе в Tauri. VFS будет отвечать за запись этих файлов в выбранное постоянное хранилище и предоставлять им уникальные идентификаторы (UUID), которые будут использоваться в дальнейшем.
2. Стратегия для промежуточных результатов (**TempResultStrategy**): Результаты промежуточных операций (например, PDF после разделения, изображение после OCR) имеют короткий жизненный цикл. Они нужны для следующего шага в цепочке обработки и больше не требуются после её

завершения. Для таких файлов VFS должна применять стратегию автоматической очистки. Это предотвращает накопление "мусора" и утечки памяти. Автоматическая очистка может быть реализована несколькими способами:

- На основе события ``beforeunload``: Перед закрытием вкладки или обновлением страницы VFS может инициировать процедуру очистки всех временных файлов.
- На основе жизненного цикла инструмента: Когда инструмент завершает свою работу и его UI-компонент отмонтируется, VFS может пометить все созданные им временные файлы как кандидатов на удаление.
- Автоматическое управление ссылками: VFS будет самостоятельно отслеживать, какие ``IFileEntry`` активны. Когда последний компонент, использующий файл, его больше не нуждается, VFS должен вызвать ``URL.revokeObjectURL`` (для временных файлов, хранящихся в памяти) или пометить файл для удаления из хранилища (для файлов в IndexedDB).

Этот механизм полностью автоматизирует управление ресурсами, избавляя разработчиков от необходимости вручную управлять `createObjectURL` и `revokeObjectURL`.

Второй важной задачей является абстракция платформы для операций ввода-вывода. Приложение должно работать как в браузере (Web), так и как десктопное приложение (Tauri). Эти две среды имеют кардинально разные модели доступа к файловой системе. Веб-приложение работает в ограниченном песочнице, используя Blob-объекты и API IndexedDB, тогда как Tauri имеет прямой доступ к нативной файловой системе устройства, что позволяет ему эффективно работать с файлами гигабайтного размера, не загружая их целиком в память ⁶⁹. Чтобы скрыть эти различия от бизнес-логики инструментов, необходимо внедрить паттерн Адаптер.

Шаблон Адаптера для Ввода-Вывода предполагает создание общего интерфейса, например, `IFileSystemAdapter`, который определит набор методов для стандартных операций с файлами:

```
interface IFileSystemAdapter {  
  readFile(fileId: string): Promise<Blob>;  
  writeFile(fileId: string, data: Blob): Promise<void>;  
  deleteFile(fileId: string): Promise<void>;  
}
```

```
// ... другие методы  
}
```

Затем создаются две реализации этого интерфейса: 1.

WebFileSystemAdapter: Эта реализация будет использовать API IndexedDB и работу с Blob-объектами для чтения и записи файлов в веб-окружении. Она будет интегрирована с VFS для управления жизненным циклом файлов. 2.

TauriFileSystemAdapter: Эта реализация будет использовать нативные API Tauri для доступа к файловой системе. Она сможет принимать на вход пути к файлам на диске, минуя Blob-объекты, что крайне эффективно для обработки больших файлов. Например, инструмент может получить путь к файлу на диске и передать его нативной библиотеке для обработки.

Выбор адаптера происходит в момент запуска приложения. В точке входа (`main.tsx` или аналогичном файле) будет происходить условная компиляция или динамический импорт, который определит, запущено ли приложение в режиме Web или Tauri, и регистрирует соответствующий адаптер в качестве реализации `IFileSystem`. Таким образом, весь код бизнес-логики, выполняемый в Web Workers, будет работать с интерфейсом `IFileSystem`, который внутри себя будет делегировать вызовы выбранному `IFileSystemAdapter`. Это позволяет легко переключать поведение приложения между двумя режимами, не меняя ни одной строки кода бизнес-логики. Такой подход соответствует принципу абстракции, который широко применяется в других областях программирования, например, в Go с библиотекой `afero` для абстракции файловой системы [20](#).

В совокупности, VFS и слой адаптеров ввода-вывода создают мощный и гибкий фундамент для управления данными в LocalPDF V6. VFS решает проблему утечек памяти и упрощает работу с файлами, предоставляя единый интерфейс. Адаптеры ввода-вывода обеспечивают кросс-платформенную совместимость, позволяя приложению эффективно использовать возможности как веб-браузера, так и десктопной среды Tauri.

Унифицированный Раннер и Декларативная Монетизация

Одной из самых сложных и трудноподдерживаемых областей в любом коммерческом приложении является логика монетизации. В текущих версиях LocalPDF эта логика размазана по коду: проверки прав доступа и ограничений (например, максимальное количество страниц в бесплатной версии) часто встречаются в разных сервисах и компонентах. Это приводит к увеличению количества проверок, что усложняет поддержку и создает риск ошибок. Запрос на создание унифицированного раннера и внедрение декларативной монетизации направлен на то, чтобы централизовать эту логику и сделать её прозрачной и легко управляемой. Концепция `V5UnifiedToolRunner` уже является хорошей отправной точкой, но она должна быть развита для полного отказа от императивной логики проверок внутри самих инструментов [1](#).

Основой декларативного подхода является определение ограничений непосредственно в метаданных инструмента. Вместо того чтобы разработчик инструмента вручную писал код вроде `if (pages > 50 && isFree) return error;`, вся информация о правилах доступа и ограничениях должна быть собрана в одном месте — в `IToolDefinition`. Это позволяет отделить описание того, что может делать инструмент, от того, как это ограничение реализуется.

Пример структуры `limits` в `IToolDefinition` может выглядеть следующим образом:

```
interface IToolDefinition {
  // ... другие поля
  limits?: {
    maxFileSize?: {
      free: number; // в байтах
      pro: number;
    };
    maxPagesPerFile?: {
      free: number;
      pro: number;
    };
    featureTier?: 'basic' | 'pro'; // Требуется Pro-подписки
    usageQuota?: { // Ежемесячный лимит использования
```

```

    free: number;
    pro: number;
  };
};
entitlements?: string[]; // RBAC rights required to use the tool
}

```

Эта структура позволяет декларативно задать широкий спектр ограничений: максимальный размер файла, количество страниц, требуемые права доступа (RBAC) и даже ежемесячные квоты на использование [53](#) [96](#). Такой подход значительно упрощает жизнь разработчикам инструментов: они больше не должны знать ничего о системе монетизации. Их задача — реализовать только основную бизнес-логику.

Унифицированный раннер становится центральным исполнителем и "полицейским" в этой архитектуре. Его задача — принимать команды на выполнение инструмента и перед тем, как запустить логику, проводить полную проверку всех ограничений. Процесс работы **UnifiedRunner** выглядит следующим образом:

1. **Получение команды:** Runner получает от **WorkerOrchestrator** команду на запуск определённого инструмента с заданными параметрами (например, список входных файлов, их размеры, количество страниц).
2. **Чтение метаданных:** Runner обращается к **GlobalRegistry**, чтобы получить **IToolDefinition** для запрашиваемого инструмента.
3. **Запрос статуса подписки:** Runner вызывает сервис **SubscriptionService** или **LimitService**, чтобы получить актуальную информацию о статусе подписки пользователя (Бесплатная или Профессиональная) и его текущем использовании (например, сколько файлов он уже обработал в этом месяце).
4. **Проверка ограничений:** Runner сравнивает переданные параметры (размер файла, количество страниц) с ограничениями, объявленными в **IToolDefinition**, и лимитами, установленными для текущего типа подписки.
5. **Принятие решения:**

- **Если ограничение превышено:** Runner не запускает Web Worker. Вместо этого он отправляет в основной поток событие, например, ``TOOL_ACCESS_DENIED``, с указанием причины (например, "Превышен лимит размера файла для бесплатной версии"). UI-компонент, подписанный на это событие, отображает модальное окно с предложением обновить подписку или выбрать другой файл.

- Если ограничения соблюдены: **Runner** собирает необходимый логический плагин (функцию обработки) и передаёт его в `WorkerOrchestrator`` для запуска в фоновом потоке.

Такой подход полностью централизует логику монетизации и прав доступа. Любые изменения в правилах (например, увеличение лимита для Pro-пользователей) требуют лишь изменения в **IToolDefinition** или в конфигурации сервиса лимитов, но не затрагивают код ни одного из инструментов. Это делает систему невероятно гибкой и легко масштабируемой. Кроме того, это позволяет легко внедрять новые уровни монетизации (например, корпоративные подписки) в будущем, просто добавив новые типы подписок в **SubscriptionService** и обновив лимиты в **IToolDefinition**.

Для управления очередями задач (очередями) также можно использовать **Runner**. Если приложение поддерживает одновременное выполнение нескольких задач, **Runner** может управлять пулом воркеров и очередью задач. Когда пользователь запрашивает выполнение инструмента, **Runner** помещает задачу в очередь. Когда освобождается воркер, **Runner** извлекает задачу из очереди и выполняет проверки, как описано выше. Если проверки пройдены, он запускает задачу. Это обеспечивает стабильную работу приложения даже при высокой нагрузке.

Внедрение декларативной монетизации соответствует современным трендам в разработке программного обеспечения, где предпочтение отдается декларативным подходам. Вместо того чтобы писать много императивного кода для описания "как" система должна действовать, мы описываем "что" система должна делать через конфигурационные данные (метаданные инструментов). Это делает систему более предсказуемой, менее подверженной ошибкам и значительно проще для разработчиков [7](#) [8](#) [49](#). Вместо того чтобы разработчик конкретного инструмента писал только функцию `(input: Blob[]) => Promise<Blob>` для воркера и UI настроек, все вопросы "можно ли этому юзеру это сделать" и "как хранить этот файл" решаются слоем платформы (**Runner** + **VFS**).

Структура Проекта и Примеры Ключевых Интерфейсов

Для успешной реализации архитектуры LocalPDF V6 необходимо чётко определить структуру проекта и определить ключевые TypeScript-интерфейсы, которые станут контрактами между различными модулями системы. Предлагаемая структура каталогов и набор интерфейсов обеспечивают модульность, читаемость и удобство поддержки, следуя лучшим практикам современной веб-разработки.

Структура каталогов нового проекта

Предлагается следующая иерархия каталогов, которая инкапсулирует логику и стили каждого инструмента, а также централизует конфигурацию и сервисы.

```
/src
├─ app/
│   ├─ components/      # Общие UI-компоненты (кнопки, модальные окна)
│   ├─ layouts/         # Макеты приложения (Header, Sidebar, MainContent)
│   └─ providers/       # React Context Providers (для GlobalRegistry, The
├─ core/
│   ├─ registry/        # Реализация GlobalRegistry
│   ├─ vfs/             # Реализация Virtual File System
│   ├─ io/              # Абстракция ввода-вывода (IFileSystem, Adapters)
│   ├─ worker/          # Реализация Worker Orchestrator
│   └─ runner/          # Реализация Unified Tool Runner
├─ plugins/
│   └─ [tool-name]/
│       ├─ ui/
│           ├─ index.tsx  # Экспорт компонента <ToolSettings />
│           └─ types.ts   # Типы для UI
│       └─ logic/
│           ├─ index.ts   # Экспорт функции main() для Worker
│           └─ types.ts   # Типы для логики
│       └─ definition.ts  # ESM-экспорт IToolDefinition
├─ services/
│   ├─ subscription/     # Сервис для работы со статусом подписки
│   └─ limits/           # Сервис для работы с лимитами
```


└─ shared/
 └─ types.ts # Общие типы для всего приложения

- ``/app``: Содержит компоненты пользовательского интерфейса и макеты страниц приложения.
- ``/core``: Ядро платформы, содержащее реализацию всех центральных сервисов: реестра, VFS, IO-адаптеров, оркестратора воркеров и раннера.
- ``/plugins``: Главный каталог для всех инструментов. Каждый инструмент является самостоятельным плагином с чётко определённой структурой (UI, логика, определение).
- ``/services``: Сервисы, которые инкапсулируют внешние зависимости, такие как API для проверки подписки.
- ``/shared/types.ts``: Централизованный файл для общих типов, чтобы избежать дублирования и обеспечить согласованность типов по всему приложению.

Примеры ключевых **TypeScript**-интерфейсов

Ниже представлены примеры кода для ключевых интерфейсов, которые определяют взаимодействие между компонентами архитектуры V6.

1. **IToolDefinition**: Определение инструмента

Этот интерфейс является ядром `GlobalRegistry` и декларативно описывает все свойства инструмента.

```
export interface IToolDefinition {  
    /**  
<ul>  
<li>Уникальный идентификатор инструмента (например, 'merge-pdf')</li>  
</ul>  
    */  
    id: string;  
  
    /**  
<ul>  
<li>Хеш для маршрутизации (например, '#/merge-pdf')</li>  
</ul>  
    */  
    hash: string;
```

```
    /**
    <ul>
    <li>Название инструмента для отображения в UI</li>
    </ul>
    */
    name: string;

    /**
    <ul>
    <li>Описание инструмента для подсказок и SEO</li>
    </ul>
    */
    description: string;

    /**
    <ul>
    <li>Права доступа (RBAC) для использования инструмента</li>
    <li>Например: ["pdf.merge", "files.upload"]</li>
    </ul>
    */
    entitlements?: string[];

    /**
    <ul>
    <li>Ограничения для данного инструмента</li>
    </ul>
    */
    limits?: {
        /**
        <ul>
        <li>Максимальный размер входного файла в байтах</li>
        <li>Может быть разным для Free и Pro подписок</li>
        </ul>
        */
        maxFileSize?: { free: number; pro: number };

        /**
        <ul>
        <li>Максимальное количество страниц в обрабатываемом файле</li>
        </ul>
        */
    }
```

```

    */
    maxPagesPerFile?: { free: number; pro: number };

    /**
<ul>
<li>Требуемый уровень подписки для доступа</li>
<li>Например, 'pro' означает, что инструмент доступен только Pro-пользоват
</ul>
    */
    featureTier?: 'basic' | 'pro';
};

    /**
<ul>
<li>Ссылка на UI-компонент инструмента (будет загружена лениво)</li>
<li>@example () => import('../plugins/split-pdf/ui')</li>
</ul>
    */
    uiLoader: () => Promise<{ default: React.ComponentType }>;

    /**
<ul>
<li>Ссылка на логическую часть инструмента (будет запущена в воркере)</li>
<li>@example () => import('../plugins/split-pdf/logic')</li>
</ul>
    */
    logicLoader: () => Promise<{ default: ToolLogicFunction }>;
}

    /**
<ul>
<li>Тип для функции, которая будет выполняться в Web Worker</li>
</ul>
    */
export type ToolLogicFunction = (payload: ToolPayload) => Promise<ToolResu

```

2. **IFileSystem:** Абстрактный интерфейс для VFS

Этот интерфейс определяет контракт для операций с файлами, предоставляемый VFS.

```
export interface IFileEntry {
  /**
  <ul>
  <li>Уникальный идентификатор файла</li>
  </ul>
  */
  readonly id: string;

  /**
  <ul>
  <li>Получить содержимое файла как Blob</li>
  </ul>
  */
  getBlob(): Promise<Blob>;

  /**
  <ul>
  <li>Получить содержимое файла как текст</li>
  </ul>
  */
  getText(): Promise<string>;

  /**
  <ul>
  <li>Получить имя файла</li>
  </ul>
  */
  getName(): string;

  /**
  <ul>
  <li>Получить размер файла в байтах</li>
  </ul>
  */
  getSize(): Promise<number>;

  /**
  <ul>
  <li>Получить тип MIME файла</li>
  </ul>
  */
}
```

```

    */
    getType(): Promise<string>;
}

export interface IFileSystem {
    /**
    <ul>
    <li>Записать новый файл в хранилище</li>
    <li>@param data Данные для записи</li>
    <li>@returns IFileEntry, представляющий записанный файл</li>
    </ul>
    */
    write(data: Blob): Promise<IFileEntry>;

    /**
    <ul>
    <li>Прочитать файл по его идентификатору</li>
    <li>@param id Идентификатор файла</li>
    <li>@returns IFileEntry</li>
    </ul>
    */
    read(id: string): Promise<IFileEntry>;

    /**
    <ul>
    <li>Удалить файл по его идентификатору</li>
    <li>@param id Идентификатор файла</li>
    </ul>
    */
    delete(id: string): Promise<void>;
}

```

3. **IWorkerMessage:** Интерфейс для обмена сообщениями с воркером

Этот интерфейс описывает структуру сообщений, передаваемых между основным потоком и Web Worker.

```

// Типы для команд, отправляемых в воркер
export type WorkerCommandPayload =
    | { type: 'PROCESS_TOOL', payload: { toolId: string; inputIds: string[];

```

```

    | { type: 'READ_FILE', payload: { fileId: string } } };

export interface IWorkerCommand {
    id: string; // Уникальный ID команды
    type: 'COMMAND';
    payload: WorkerCommandPayload;
}

// Типы для событий, отправляемых из воркера
export type WorkerEventPayload =
    | { type: 'PROGRESS', payload: { progress: number } }
    | { type: 'RESULT', payload: { outputIds: string[] } }
    | { type: 'ERROR', payload: { message: string } };

export interface IWorkerEvent {
    id: string; // ID команды, которую это событие завершило
    type: 'EVENT';
    payload: WorkerEventPayload;
}

```

Эти интерфейсы формируют прочную основу для архитектуры V6. Они обеспечивают строгую типизацию, что снижает вероятность ошибок и делает код более читаемым и поддерживаемым [17](#). Разработчик инструмента видит четкие контракты: он знает, какую функцию нужно экспортировать для воркера, и как с файлами взаимодействовать через VFS. Разработчик платформы видит, как эти интерфейсы используются в раннере и оркестраторе для централизованного управления.

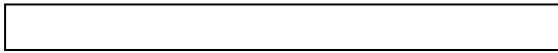
Анализ и Синтез: Полный Цикл Операции и Итоговые Рекомендации

Для завершения технической спецификации необходимо проиллюстрировать работу всей архитектуры на примере полного цикла операции. Рассмотрим сценарий: "Пользователь выбрал 5 файлов -> Объединить PDF -> Скачивание". Этот пример продемонстрирует взаимодействие всех ключевых компонентов V6: от выбора файлов в пользовательском интерфейсе до получения результата и его сохранения.

Текстовое описание потока данных:

1. Выбор файлов пользователем:

- Пользователь открывает приложение, переходит в раздел "Объединить PDF" и выбирает 5 PDF-файлов в диалоговом окне браузера.
- UI-компонент ``MergeToolUI`` (который был лениво загружен через ``React.lazy`` при переходе по маршруту ``/merge-pdf``) получает ``FileList`` от обработчика события ```



```.

- Компонент ``MergeToolUI`` инициализирует ``VirtualFileSystem`` (VFS) и вызывает его метод ``write()`` для каждого выбранного файла. VFS, опираясь на ``Storage Strategy``, сохраняет эти файлы в долгосрочное хранилище (например, IndexedDB для Web). За каждый сохраненный файл VFS возвращает объект ``IFileEntry``, который содержит уникальный идентификатор файла (UUID). UI-компонент сохраняет эти идентификаторы в своём состоянии.

### 1. Инициация операции:

- Пользователь нажимает кнопку "Объединить".
- ``MergeToolUI`` собирает массив идентификаторов полученных файлов и вызывает сервисный метод, например, ``toolRunnerService.startMerge({ fileIds: ['uuid-1', 'uuid-2', ...] })``.

### 1. Проверка и запуск в раннере:

- ``toolRunnerService`` передает полученную команду в центральный ``UnifiedToolRunner``.
- ``UnifiedToolRunner`` обращается к ``GlobalRegistry``, чтобы получить ``IToolDefinition`` для инструмента "merge-pdf".
- Runner читает секцию ``limits`` из ``IToolDefinition``. Допустим, он видит ограничение ``maxFileSize: { free: 50 1024 1024, pro: 2000 1024 1024 }``.
- Runner вызывает ``SubscriptionService``, чтобы узнать статус подписки пользователя и его текущие лимиты.
- Runner итерируется по списку ``fileIds``, обращаясь к VFS для получения информации о каждом файле (например, его размер). Он проверяет, не превышает ли размер хотя бы одного файла установленный лимит для текущего типа подписки.
- Если лимит соблюдается, Runner продолжает. Если нет, он немедленно прекращает выполнение, отправляет в основной поток событие



``TOOL_ACCESS_DENIED`` с сообщением об ошибке, и UI отображает модальное окно с предложением обновить подписку. В этом случае выполнение останавливается.

#### 1. Запуск воркера:

- Поскольку все проверки пройдены, ``UnifiedToolRunner`` вызывает ``WorkerOrchestrator``.
- ``WorkerOrchestrator`` создает новый Web Worker. Он загружает логический плагин для инструмента "merge-pdf" (который был указан в ``logicLoader`` в ``IToolDefinition``) и передает ему идентификаторы входных файлов.
- Внутри воркера запускается функция, экспортируемая из ``logic/index.ts``. Эта функция получает идентификаторы файлов.

#### 1. Обработка в воркере:

- Функция в воркере использует ``IFileSystem`` (который является обёрткой над ``IO Adapter``). Она последовательно читает каждый входной файл по его идентификатору, получая их содержимое в виде ``Blob``.
- Она использует библиотеку ``pdf-lib`` для чтения содержимого каждого PDF, копирования страниц и объединения их в один новый документ.
- После создания объединённого PDF, она снова использует ``IFileSystem.write()`` для сохранения результата. VFS возвращает новый ``IFileEntry`` для созданного файла.
- Для передачи больших Blob между основным потоком и воркером следует использовать Transferable Objects [90](#).

#### 1. Возврат результата и обновление UI:

- Функция в воркере завершает свою работу и отправляет в основной поток событие ``TOOL_RESULT`` с массивом идентификаторов выходных файлов (в данном случае, ``[newUuid]``).
- ``WorkerOrchestrator`` перехватывает это событие и передает его дальше, например, в ``toolRunnerService``.
- ``toolRunnerService`` сообщает UI-компоненту ``MergeToolUI``, что операция завершена успешно.
- ``MergeToolUI`` получает новый идентификатор файла из события и может либо отобразить его в списке результатов, либо предоставить пользователю кнопку для скачивания.
- Когда пользователь нажимает кнопку "Скачать", UI вызывает ``vfs.read(newUuid)`` и использует стандартный механизм браузера для

скачивания файла (например, создавая временную ссылку и вызывая `click()` на `` элементе).

Этот подробный сценарий показывает, как все элементы архитектуры V6 работают вместе как единая, слаженная система. Каждый компонент выполняет свою чётко определённую роль: **GlobalRegistry** предоставляет конфигурацию, плагиновая архитектура обеспечивает модульность, **Worker Orchestrator** гарантирует производительность, **VFS** управляет данными и памятью, а **Unified Runner** централизует логику монетизации.

Итоговые рекомендации:

1. Принять гибридную плагинную модель: Разделить UI и бизнес-логику. UI-компоненты должны загружаться лениво через ES-модули, а вся ресурсоёмкая логика — изолироваться в Web Workers. Это фундаментальный сдвиг от Legacy-подхода.
2. Реализовать Глобальный Реестр: Создать центральный источник истины в виде набора метаданных инструментов (**IToolDefinition**). Использовать его для динамической маршрутизации, построения навигации и управления правами доступа.
3. Разработать Виртуальную Файловую Систему (**VFS**): Внедрить VFS с единым интерфейсом **IFileEntry** и разными стратегиями хранения для постоянных и временных файлов. Это критически важно для предотвращения утечек памяти и упрощения работы с данными.
4. Внедрить слой Абстракции Ввода-Вывода (**IO Adapter**): Создать интерфейс **IFileSystemAdapter** с реализацией для Web (IndexedDB) и Tauri (нативная FS), чтобы обеспечить кросс-платформенную совместимость и эффективность работы с большими файлами.
5. Централизовать логику монетизации: Разработать **UnifiedToolRunner**, который будет считывать декларативные ограничения (**limits**) из **IToolDefinition** и проверять их до запуска любой задачи. Это сделает систему монетизации прозрачной, гибкой и легко поддерживаемой.
6. Использовать стандартизированный протокол коммуникации: Реализовать паттерн Command/Event для обмена сообщениями между UI и воркерами, а также использовать передаваемые объекты для эффективной передачи больших данных.

Следование этим рекомендациям позволит создать архитектуру LocalPDF V6, которая не только решит накопленные проблемы старых версий, но и станет прочным, масштабируемым и технологически современным фундаментом для будущего развития продукта.

---

## Справка

1. [Oracle Database 19c Upgrade Guide](https://pdfcoffee.com/oracle-database-19c-upgrade-guide-pdf-free.html) <https://pdfcoffee.com/oracle-database-19c-upgrade-guide-pdf-free.html>
2. [How can I add external application bundle in my Tauri app](https://stackoverflow.com/questions/78357312/how-can-i-add-external-application-bundle-in-my-tauri-app) <https://stackoverflow.com/questions/78357312/how-can-i-add-external-application-bundle-in-my-tauri-app>
3. [Awesome Angular](https://gitee.com/awesome-lib/awesome-angular) <https://gitee.com/awesome-lib/awesome-angular>
4. [The OpenHands Software Agent SDK: A Composable and ...](https://arxiv.org/html/2511.03690v1) <https://arxiv.org/html/2511.03690v1>
5. [Exploring Blockchain Technology through a Modular Lens](https://dl.acm.org/doi/full/10.1145/3657288) <https://dl.acm.org/doi/full/10.1145/3657288>
6. [Pages llms-full.txt](https://developers.cloudflare.com/pages/llms-full.txt) <https://developers.cloudflare.com/pages/llms-full.txt>
7. [Declarative agent schema 1.2 for Microsoft 365 Copilot](https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-manifest-1.2) <https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-manifest-1.2>
8. [Declarative Models for Ubiquitous Web Applications](https://www.w3.org/2007/02/dmdwa-ws/Papers/jose-m-c-fonseca.html) <https://www.w3.org/2007/02/dmdwa-ws/Papers/jose-m-c-fonseca.html>
9. [Declarative Gradle vs Maven: A Build Tool Comparison](https://www.linkedin.com/posts/joshlong_declarative-gradle-eap3-april-2025-update-activity-7322929514903207936-Dtkv) [https://www.linkedin.com/posts/joshlong\\_declarative-gradle-eap3-april-2025-update-activity-7322929514903207936-Dtkv](https://www.linkedin.com/posts/joshlong_declarative-gradle-eap3-april-2025-update-activity-7322929514903207936-Dtkv)
10. [Declarative vs Programmatically GraphQL](https://stackoverflow.com/questions/45390237/declarative-vs-programmatically-graphql) <https://stackoverflow.com/questions/45390237/declarative-vs-programmatically-graphql>
11. [Declarative Setup - Argo CD - Read the Docs](https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/) <https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/>
12. [URL.createObjectURL和URL.revokeObjectURL必须成对 ...](https://blog.csdn.net/liuzehn/article/details/105449974) <https://blog.csdn.net/liuzehn/article/details/105449974>

13. Web Worker 常见使用问题和解决方案 - 腾讯云 <https://cloud.tencent.com/developer/article/2395000>
14. Bug List <https://www.w3.org/Bugs/Public/buglist.cgi?product=WebAppsWG>
15. Sharing variables between web workers? [global variables?] <https://stackoverflow.com/questions/2262681/sharing-variables-between-web-workers-global-variables>
16. Moving away from ReactJs and VueJs on front-end using ... <https://dev.to/xurxodev/moving-away-from-reactjs-and-vuejs-on-front-end-using-clean-architecture-3olk?ref=xurxodev.com>
17. "5 TypeScript Prompts for AI Coding Assistants" [https://www.linkedin.com/posts/aliajafri\\_unlocking-the-power-of-prompt-engineering-activity-7382129841170714624-xJE8](https://www.linkedin.com/posts/aliajafri_unlocking-the-power-of-prompt-engineering-activity-7382129841170714624-xJE8)
18. 计算机知识梳理思维导图模板 <https://www.processon.com/view/64448a8d0fb07e723417ffa7>
19. TowardsDataScience 博客中文翻译2019（一百零七） 原创 <https://blog.csdn.net/wizardforcel/article/details/142536513>
20. 平台和工具 - 架构师研究会 <https://architect.pub/book/export/html/36>
21. MDN 0214DG PDF <https://www.scribd.com/document/410621715/MDN-0214DG-pdf>
22. Lazy load page components vs ES6 Imports <https://stackoverflow.com/questions/78482268/lazy-load-page-components-vs-es6-imports>
23. Why I Decided to Stop Working with React.js in 2025 <https://dev.to/holasoymalva/why-i-decided-to-stop-working-with-reactjs-in-2025-4d1l>
24. How to pass refs to lazy load components <https://stackoverflow.com/questions/78901606/how-to-pass-refs-to-lazy-load-components>
25. Newest 'webpack-module-federation' Questions <https://stackoverflow.com/questions/tagged/webpack-module-federation?tab=Newest>
26. Published Password Lists: 1 [https://ineapple.com/known\\_pass1](https://ineapple.com/known_pass1)
27. The application may be doing too much work on its main ... <https://stackoverflow.com/questions/14678593/the-application-may-be-doing-too-much-work-on-its-main-thread>
28. How Web Workers Improve UI Performance [https://www.linkedin.com/posts/niket-chandivade-576864137\\_unblocking-the-ui-with-web-workers-a-guide-activity-7384544132985249792-0HKK](https://www.linkedin.com/posts/niket-chandivade-576864137_unblocking-the-ui-with-web-workers-a-guide-activity-7384544132985249792-0HKK)
29. 1 Introduction <https://arxiv.org/html/2508.15105v3>
30. Construction and Application of Digital Twin Cloud Platform ... <https://dl.acm.org/doi/fullHtml/10.1145/3661638.3661690>

31. Industry 4.0 smart reconfigurable manufacturing machines <https://www.sciencedirect.com/science/article/pii/S027861252100056X>
32. Compilation and Build-DevEco Studio-Develop <https://developer.huawei.com/consumer/en/doc/harmonyos-faqs/faqs-compiling-and-building-4>
33. Boot failing in 24.04 (after working fine for a few weeks) <https://discourse.ubuntu.com/t/boot-failing-in-24-04-after-working-fine-for-a-few-weeks/54234>
34. Tree Shaking <https://webpack.js.org/guides/tree-shaking/>
35. Unity 2023.3.0b9 <https://unity.com/releases/editor/beta/2023.3.0b9>
36. Spellbook of Modern Web Dev <https://bestofjs.org/projects/spellbook-of-modern-web-dev>
37. crates.io/crates - SJTUG Mirror Index [https://s3.jcloud.sjtu.edu.cn/899a892efef34b1b944a19981040f55b-oss01/crates.io/crates/mirror\\_clone\\_list.html](https://s3.jcloud.sjtu.edu.cn/899a892efef34b1b944a19981040f55b-oss01/crates.io/crates/mirror_clone_list.html)
38. 解析Web Worker <https://juejin.cn/post/7233326040088510521>
39. blob: URLs - URIs - MDN Web Docs <https://developer.mozilla.org/en-US/docs/Web/URI/Reference/Schemes/blob>
40. 前端魔法堂：可能是你见过最详细的WebWorker实用指南 <https://www.cnblogs.com/fsjohnhuang/p/14141311.html>
41. PyCharm 181.4203.547 Release Notes | Knowledge Base <https://youtrack.jetbrains.com/articles/PY-A-113869059/PyCharm-181-4203-547-Release-Notes>
42. ie-20221231 <https://www.sec.gov/Archives/edgar/data/1879016/000187901623000003/ie-20221231.htm>
43. Full article: Structural Racism in the COVID-19 Pandemic <https://www.tandfonline.com/doi/full/10.1080/15265161.2020.1851808>
44. THE ALCOHOL AND OTHER DRUG TREATMENT COURT ... [https://www.researchgate.net/profile/Toni\\_Carr/publication/340775407\\_Governing\\_addiction\\_The\\_Alcohol\\_and\\_Other\\_Drug\\_Treatment\\_Court\\_in\\_New\\_Zealand/links/5ee2a797299bf1faac4e5df9/Governing-addiction-The-Alcohol-and-Other-Drug-Treatment-Court-in-New-Zealand.pdf](https://www.researchgate.net/profile/Toni_Carr/publication/340775407_Governing_addiction_The_Alcohol_and_Other_Drug_Treatment_Court_in_New_Zealand/links/5ee2a797299bf1faac4e5df9/Governing-addiction-The-Alcohol-and-Other-Drug-Treatment-Court-in-New-Zealand.pdf)
45. Model Context Protocol servers <https://gitee.com/mirrors/Model-Context-Protocol>
46. EBOOK 100 AI Startups | PDF | Artificial Intelligence <https://www.scribd.com/document/744464581/EBOOK-100-AI-Startups>
47. D4.1 Integration guidelines & initial NEMO integration <https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e507720535&appId=PPGMS>

48. AZURE Archives | Page 2 of 5 <https://www.cloudiqtech.com/category/blogs/azure/page/2/>
49. Computer Science <https://arxiv.org/list/cs/new>
50. Fusion Cloud Services <https://www.oracle.com/contracts/docs/oracle-fusion-cloud-service-desc-1843611.pdf>
51. Service Workers Nightly <https://www.w3.org/TR/service-workers/>
52. TENDER SPECIFICATIONS [https://ec.europa.eu/info/funding-tenders/opportunities/portal/screen/opportunities/tender-details/docs/161847b4-b0cc-4fb0-84aa-3d0286b7a0a7-CN/MN5\\_AI\\_Upgrade-Technical\\_Specifications\\_Part\\_2%20Version%203\\_V3.pdf](https://ec.europa.eu/info/funding-tenders/opportunities/portal/screen/opportunities/tender-details/docs/161847b4-b0cc-4fb0-84aa-3d0286b7a0a7-CN/MN5_AI_Upgrade-Technical_Specifications_Part_2%20Version%203_V3.pdf)
53. Azure subscription and service limits, quotas, and constraints <https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-subscription-service-limits>
54. \ourbench: A Multi-Round Benchmark for Assessing the ... <https://arxiv.org/html/2502.12904v2>
55. EMQX Enterprise Version 5 <https://docs.emqx.com/en/emqx/latest/changes/changes-ee-v5.html>
56. Chapter 2. Container security | Security and compliance [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/security\\_and\\_compliance/container-security-1](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/security_and_compliance/container-security-1)
57. Business Products <https://docs.daocloud.io/native/commerce/>
58. how to pass large data to web workers <https://stackoverflow.com/questions/19152772/how-to-pass-large-data-to-web-workers>
59. Worker threads | Node.js v25.6.0 Documentation [https://nodejs.org/api/worker\\_threads.html](https://nodejs.org/api/worker_threads.html)
60. Development & testing · Cloudflare Workers docs <https://developers.cloudflare.com/workers/development-testing/>
61. 通用软件项目技术报告- 术语词典原创 [https://blog.csdn.net/m0\\_61940795/article/details/147962902](https://blog.csdn.net/m0_61940795/article/details/147962902)
62. Crash Roadmap | PDF | Html Element <https://www.scribd.com/document/931572126/Crash-Roadmap>
63. Track Awesome Go Updates Daily <https://www.trackawesomelist.com/abordage/awesome-go/>
64. hiifong/starList: Export your star's repository list <https://gitea.com/hiifong/starList>
65. Cektile | PDF | Microsoft Sql Server <https://www.scribd.com/document/752533028/cektile>

66. Awesome List Updates on Nov 24, 2025 <https://www.trackawesomelist.com/2025/11/24/>
67. Failed to fetch dynamically imported module" on Vue/Vite ... <https://stackoverflow.com/questions/69300341/typeerror-failed-to-fetch-dynamically-imported-module-on-vue-vite-vanilla-set>
68. (PDF) Improving Front-end Performance through Modular ... [https://www.researchgate.net/publication/390570785\\_Improving\\_Front-end\\_Performance\\_through\\_Modular\\_Rendering\\_and\\_Adaptive\\_Hydration\\_MRAH\\_in\\_React\\_Applications](https://www.researchgate.net/publication/390570785_Improving_Front-end_Performance_through_Modular_Rendering_and_Adaptive_Hydration_MRAH_in_React_Applications)
69. Full-Stack Offline App Development | PDF | Java Script <https://www.scribd.com/document/864860418/Full-Stack-Offline-App-Development>
70. Linknovate | Profile for GitHub <https://www.linknovate.com/affiliation/github-435897/all/?query=unlimited%20email%20validation%20potential>
71. How to transfer large objects using postMessage of ... <https://stackoverflow.com/questions/34057127/how-to-transfer-large-objects-using-postmessage-of-webworker>
72. HTTP messages - MDN Web Docs - Mozilla <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>
73. Media Source Extensions™ <https://www.w3.org/TR/media-source-2/>
74. Durable Objects llms-full.txt <https://developers.cloudflare.com/durable-objects/llms-full.txt>
75. Design Patterns for Humans: Making Complex Software ... <https://www.linkedin.com/pulse/design-patterns-humans-making-complex-software-simple-abdulmoiz-ahmer-ol4uf>
76. Modern Slavery, Victim Identification and the 'Victimized State' <https://academic.oup.com/bjc/article/65/3/504/7738359>
77. Restorative Justice and Responsive Regulation <http://johnbraithwaite.com/wp-content/uploads/2019/02/Restorative-Justice-and-Responsive-regulation-book.pdf>
78. NodeJS Fundamentals: dynamic import <https://dev.to/devopsfundamentals/nodejs-fundamentals-dynamic-import-lbd>
79. Lazy loading - Performance - MDN Web Docs - Mozilla [https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Lazy\\_loading](https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Lazy_loading)
80. Advanced JavaScript Modules Explained: Building Clean ... <https://www.linkedin.com/pulse/advanced-javascript-modules-explained-building-clean-reusable-r-7ngcc>
81. Mysterious "Not enough quota is available to process this ... <https://stackoverflow.com/questions/12584619/mysterious-not-enough-quota-is-available-to-process-this-command-in-winrt-port>

82. A Client-Side Security Assessment Framework <https://www.arxiv.org/pdf/2505.08050>
83. Workers llms-full.txt <https://developers.cloudflare.com:2053/workers/llms-full.txt>
84. Lightweight GaussDB 25.1.32 Usage Guide 01 <https://support.huawei.com/enterprise/de/doc/EDOC1100484823?currentPartNo=k00e&togo=content>
85. Chapter 14. Performance Addon Operator for low latency ... [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.9/html/scalability\\_and\\_performance/cnf-performance-addon-operator-for-low-latency-nodes](https://docs.redhat.com/en/documentation/openshift_container_platform/4.9/html/scalability_and_performance/cnf-performance-addon-operator-for-low-latency-nodes)
86. Amazon Elastic Container Service - Best Practices Guide <https://docs.aws.amazon.com/pdfs/AmazonECS/latest/bestpracticesguide/bestpracticesguide.pdf>
87. Obsidian for the web with VSCode extension <https://forum.obsidian.md/t/obsidian-for-the-web-with-vscode-extension/81918>
88. Trinity Continuum - Anima (OEF) (2023!06!27) - iT5iJ7 | PDF <https://www.scribd.com/document/693917799/Trinity-Continuum-Anima-OEF-2023-06-27-iT5iJ7>
89. ANGLIA RUSKIN UNIVERSITY FACULTY OF ARTS, ... [https://s3-eu-west-1.amazonaws.com/pstorage-arx-9188083843/59203871/Mann\\_2025.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIA3OGA3B5WBW2MQZ3S/20251127/eu-west-1/s3/aws4\\_request&X-Amz-Date=20251127T201451Z&X-Amz-Expires=86400&X-Amz-SignedHeaders=host&X-Amz-Signature=2ffb00b637b845e5852ffb28cd2b0e3544853447eed5d616479da759af06369b](https://s3-eu-west-1.amazonaws.com/pstorage-arx-9188083843/59203871/Mann_2025.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIA3OGA3B5WBW2MQZ3S/20251127/eu-west-1/s3/aws4_request&X-Amz-Date=20251127T201451Z&X-Amz-Expires=86400&X-Amz-SignedHeaders=host&X-Amz-Signature=2ffb00b637b845e5852ffb28cd2b0e3544853447eed5d616479da759af06369b)
90. Web Workers in Javascript | PDF <https://www.scribd.com/document/863584918/2-1>
91. Harnessing Parallel Computing in Node.js: Algorithms, ... <https://www.linkedin.com/pulse/harnessing-parallel-computing-nodejs-algorithms-data-structures-r-eep0c>
92. Declarative agent schema 1.0 for Microsoft 365 Copilot <https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-manifest-1.0>
93. Declarative agent schema 1.6 for Microsoft 365 Copilot <https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-manifest-1.6>
94. API Security and Management <https://docs.broadcom.com/docs/kuppingercole-leadership-compass-api-security-and-management-2025>
95. Hybrid Resource Quota Scaling for Kubernetes-Based ... <https://www.mdpi.com/2079-9292/14/16/3308>
96. Amazon Bedrock endpoints and quotas <https://docs.aws.amazon.com/general/latest/gr/bedrock.html>
97. Providing Realtime Support for Containerized Edge Services <https://dl.acm.org/doi/10.1145/3617123>



98. Democratic Republic of Congo <https://documents1.worldbank.org/curated/en/171101529346675751/pdf/DRC-SCD-FINAL-ENGLISH-06132018.pdf>
99. View and manage logs | Adobe Commerce - Experience League <https://experienceleague.adobe.com/en/docs/commerce-on-cloud/user-guide/develop/test/log-locations>
100. Using Web Workers - Web APIs - MDN Web Docs [https://developer.mozilla.org/en-US/docs/Web/API/Web\\_Workers\\_API/Using\\_web\\_workers](https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers)
101. React Performance Optimization: 15 Best Practices for 2025 [https://dev.to/alex\\_bobes/react-performance-optimization-15-best-practices-for-2025-1719](https://dev.to/alex_bobes/react-performance-optimization-15-best-practices-for-2025-1719)
102. Transfer file to webworker: DataCloneError: The object ... <https://stackoverflow.com/questions/26055730/transfer-file-to-webworker-datacloneerror-the-object-could-not-be-cloned>
103. Web Worker通信中DataCloneError错误分析与解决方案 <https://comate.baidu.com/zh/page/6qmfzrbq5tm>
104. Technologies | January 2026 - Browse Articles <https://www.mdpi.com/2227-7080/14/1>
105. A Survey of Vibe Coding with Large Language Models <https://arxiv.org/html/2510.12399v1>
106. Cron Jobs vs Real Task Schedulers: A Love Story <https://dev.to/elvissautet/cron-jobs-vs-real-task-schedulers-a-love-story-1fka>
107. create something I can paste into p5js that will startle me ... [https://www.linkedin.com/posts/emollick\\_i-gave-most-of-the-frontier-models-this-prompt-activity-7274287006556250112-QB\\_8](https://www.linkedin.com/posts/emollick_i-gave-most-of-the-frontier-models-this-prompt-activity-7274287006556250112-QB_8)
108. Configuration Macros - HTCondor Manual - Read the Docs <https://htcondor.readthedocs.io/en/25.x/admin-manual/configuration-macros.html>
109. Release Notes | Red Hat OpenStack Platform | 12 [https://docs.redhat.com/en/documentation/red\\_hat\\_openshift\\_platform/12/html-single/release\\_notes/index](https://docs.redhat.com/en/documentation/red_hat_openshift_platform/12/html-single/release_notes/index)
110. VMware NSX 4.2.0 Release Notes - Broadcom Techdocs <https://techdocs.broadcom.com/us/en/vmware-cis/nsx/vmware-nsx/4-2/release-notes/vmware-nsx-420-release-notes.html>